



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

File

Angelo Sotgiu

angelo.sotgiu@unica.it

In questa lezione

- async - await
- Download e Upload di file



async -await

- Quando bisogna usare async nella definizione delle funzioni di endpoint?
FastAPI gestisce autonomamente la maggior parte dei casi, quindi suggeriscono di:
 - usare `async def` se dobbiamo richiamare funzioni che necessitano `await` per comunicare con un sistema esterno
 - usare `def` se dobbiamo richiamare funzioni che non supportano `await` per comunicare con un sistema esterno
 - usare `async def` se la funzione non comunica con nessun sistema esterno
 - usare `def` in caso di dubbio (`async def` spesso causa rallentamenti)
- Nella nostra applicazione, è giusto usare `def` nelle funzioni che interagiscono con il DB (in pratica tutte). Possiamo aggiungere `async` nelle funzioni `home` e `add_book_form` (non cambierà nulla)



Download di File

- Vogliamo aggiungere due nuove funzionalità per fare in modo che dalla pagina web con la lista dei libri:
 - cliccando su un certo libro si scarichi un JSON contenente le informazioni di quel libro
 - cliccando su un bottone si possa scaricare il file del database
- Abbiamo bisogno di:
 - un nuovo endpoint che restituisca il JSON del libro richiesto `/books/{id}/download`
 - un bottone-link per ogni libro che rimandi al nuovo endpoint con il parametro corretto
 - un nuovo endpoint che restituisca il file del database
 - un bottone-link che avvii il download
- Ovviamente, dovremmo convertire l'oggetto Book in un JSON. Dato che si tratta di un oggetto volatile in memoria (e non di un file persistente), possiamo caricarlo su un buffer

API GET /books/{id}/download

```
from fastapi.responses import StreamingResponse
from io import BytesIO

@router.get("/{id}/download", response_class=StreamingResponse)
async def download_book(
    session: SessionDep,
    id: Annotated[int, Path(description="The ID of the book to delete")]
) -> StreamingResponse:
    """Returns as a JSON file the book with the given ID"""
    book = session.get(Book, id)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")
    buffer = BytesIO(book.model_dump_json().encode("utf-8"))
    headers = {"Content-Disposition": f"attachment; filename={book.title}.json"}
    return StreamingResponse(buffer, headers=headers,
                             media_type="application/octet-stream")
```

Template list.html

- Modifico solo la riga della tabella che viene creata per ogni libro. Uso un elemento form per poter effettuare una richiesta GET quando premo il pulsante.

```
{% for book in books %}
<tr>
  <td>{{ book.id }}</td>
  <td>{{ book.title }}</td>
  <td>{{ book.author }}</td>
  <td>{{ book.review }}</td>
  <td>
    <form action="{{ url_for('download_book', id=book.id) }}" method="get">
      <input type="submit" value="Download" />
    </form>
  </td>
</tr>
```

API GET /books/download_db

```
from data.db import SessionDep, sqlite_file_name
from fastapi.responses import StreamingResponse, FileResponse

@router.get("/download_db", response_class=FileResponse)
async def download_db() -> FileResponse:
    """Returns the DB file"""
    headers = {"Content-Disposition": f"attachment; filename=database.db"}
    return FileResponse(sqlite_file_name, headers=headers)
```

- Ricordiamo di aggiungere questa funzione PRIMA di quelle parametriche `/ {id} ...`
- Quando usare StreamingResponse: file generati dinamicamente, buffer, file di grandi dimensioni
- Quando usare FileResponse: file standard di piccole dimensioni

Template list.html

- Dopo la tabella, prima di {% endblock %}

```
<br>
<form action="{ { url_for('download_db') } }" method="get" style="text-align:
  center">
  <input type="submit" value="Download DB" />
</form>
{% endblock %}
```


Upload di File

- Proviamo ora ad aggiungere un libro dal DB tramite upload di un file JSON.
Dobbiamo scrivere:
 - un nuovo endpoint che riceva il file, ne validi il contenuto e aggiunga il file al DB
 - un nuovo bottone che consenta il caricamento del file, nella pagina web "Add Book"
- Iniziamo dal bottone, nel template add.html (dopo il form già presente)

```
<br>  
<form action="{ url_for('add_book_from_file') }" method="post"  
      style="text-align: center" enctype="multipart/form-data">  
  <input type="file" accept=".json" name="file" id="file" required>  
  <input type="submit" value="Upload JSON" />  
</form>  
{% endblock %}
```

API POST /books_file

```
@router.post("_file/")
async def add_book_from_file(
    session: SessionDep,
    file: UploadFile,
):
    """Adds a new book."""
    try:
        book = await file.read()
        session.add(Book.model_validate_json(book))
        session.commit()
        return "Book successfully added"
    except:
        raise HTTPException(400, detail="Invalid file")
```

